



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 3. Коллекции

Содержание лекции

В данной главе будут рассмотрены следующие темы:

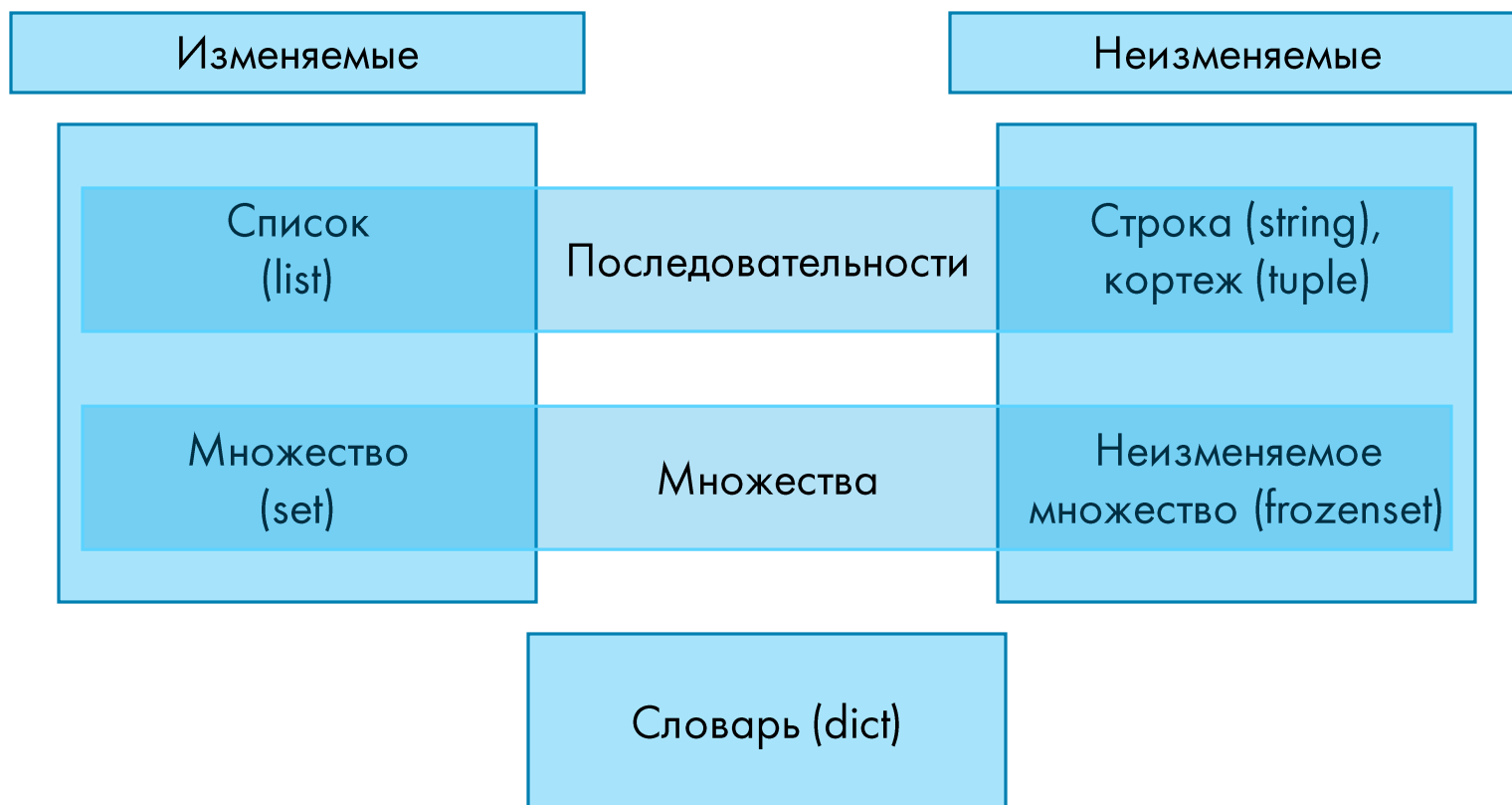
-  Классификация коллекций
-  Последовательности
-  Множества
-  Словари
-  Стандартные функции
-  Сортировка
-  Индексирование
-  Срезы
-  Модуль collections

Классификация коллекций

📶 Коллекция – объект, хранящий набор значений

- 🕒 Значения могут быть разных типов

📶 Виды коллекций:



Последовательности (1 из 2)

- 📡 Список – индексированная, изменяемая последовательность значений произвольных типов
 - ⦿ Каждый элемент расположен в нумерованной позиции
 - ⦿ Элементы можно удалять, добавлять и изменять
 - ⦿ Элементы могут быть любого типа
- 📡 Элементы списка заключены в квадратные скобки []

```
list_1 = []  
list_2 = ['H', 'e', 'l', 'l', 'o']  
list_3 = list("Hello")  
list_4 = [letter for letter in "Hello"]
```

Последовательности (2 из 2)

- 📡 Кортеж – индексированная, неизменяемая последовательность значений произвольных типов
 - ⦿ Каждый элемент расположен в нумерованной позиции
 - ⦿ Элементы **нельзя** удалять, добавлять и изменять
 - ⦿ Элементы могут быть любого типа

```
tuple_1 = ()
```

```
tuple_wrong = (1)
```

```
print(type(tuple_wrong)) Вывод: <class 'int'>
```

```
tuple_correct = (1,)
```

```
print(type(tuple_correct)) Вывод: <class 'tuple'>
```

```
tuple_2 = ('H', 'e', 'l', 'l', 'o')
```

```
tuple_3 = tuple("Hello")
```

```
tuple_4 = tuple([letter for letter in "Hello"])
```

Множества (1 из 2)

- Множество – неиндексированный, изменяемый набор уникальных значений произвольных типов
 - Элементы расположены в случайном порядке
 - Элементы можно удалять, добавлять и изменять
 - Элементы могут быть любого типа

```
set_1 = set()
set_2 = {'H', 'e', 'l', 'l', 'o'}
set_3 = set("Hello")
set_4 = {letter for letter in "Hello"}

set_3.add('w')

print(set_3)
Вывод: {'e', 'H', 'w', 'o', 'l'}
```

Случайный порядок
уникальных значений

Множества (2 из 2)

- ❶ Неизменяемое множество - неиндексированный, неизменяемый набор уникальных значений произвольных типов
 - ⦿ Элементы расположены в случайном порядке
 - ⦿ Элементы **нельзя** удалять, добавлять и изменять
 - ⦿ Элементы могут быть любого типа

```
frozenset_1 = frozenset()
frozenset_2 = frozenset("Hello")
frozenset_3 = frozenset(
    {letter for letter in "Hello"}
)

frozenset_2.add("World")
```

Вызовет ошибку, так как это неизменяемое множество

Словари

 Словарь – неиндексированный набор пар ключ:значение

- ⦿ Ключи уникальны; значения могут повторяться
- ⦿ Ключи менять нельзя; значения можно менять
- ⦿ Ключом может быть любой неизменяемый тип
- ⦿ Значение может быть любого типа

 Словари часто называют ассоциативными массивами

```
dict_1 = {}  
dict_2 = dict(first='Hello', second='world!')  
dict_3 = {  
    number: letter  
    for number, letter  
    in enumerate("Hello", 1)  
}  
print(dict_3) Вывод: {1: 'H', 2: 'e', 3: 'l', 4: 'l', 5: 'o'}
```


Стандартные функции

📡 Ряд функций применяется к коллекциям всех типов

- ⦿ `print()` – вывод всех элементов коллекции
- ⦿ `len()` – возвращает количество элементов коллекции
- ⦿ Оператор `"in"` – возвращает `True`, если элемент входит в коллекцию
- ⦿ Цикл `for ... in ...` для обхода коллекции
- ⦿ Функции `max()`, `min()`, `sum()`

```
dict_1 = {1: "a", 2: "b", 3: "c"}
```

```
print("b" in dict_1)
```

Вывод: False

```
for elem in dict_1:
```

```
    print(elem)
```

Вывод: 1 2 3

```
print(max(dict_1))
```

Вывод: 3

По умолчанию в словарях перебираются ключи

Сортировка

- 📡 Функция `sorted()` возвращает список из отсортированных элементов переданной коллекции
- 📡 Сортировка словарей:
 - 🕒 `sorted(dict_1)` — возвращает отсортированный список ключей
 - 🕒 `sorted(dict_1.keys())` — аналогично `sorted(dict_1)`
 - 🕒 `sorted(dict_1.items())` — возвращает отсортированный список кортежей (ключ, значение)
 - 🕒 `sorted(dict_1.values())` — возвращает отсортированный список значений
- 📡 У списков есть метод `.sort()`
 - 🕒 Меняет исходный список и возвращает `None`

```
dict_1 = {1: 'a', 3: 'c', 2: 'b', 4: 'd'}
```

```
print(sorted(dict_1.items()))
```

Вывод: [(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd')]

Индексирование (1 из 2)

- В индексированных коллекциях каждый элемент имеет свой индекс
- Положительные индексы:
 - Индекс первого элемента коллекции: 0
 - Индекс последнего: Количество элементов - 1
- Отрицательные индексы:
 - Индекс последнего элемента коллекции: -1
 - Индекс первого: -Количество элементов

-6	-5	-4	-3	-2	-1
S	T	R	I	N	G
0	1	2	3	4	5

Индексирование (2 из 2)

- 📡 Получение элемента по индексу доступно только для индексированных коллекций
- 📡 Попытка получить элемент по несуществующему индексу приведет к ошибке
 - 🕒 Возникнет ошибка `IndexError`
- 📡 Попытка получить по индексу элемент неиндексированной коллекции также приведет к ошибке
 - 🕒 Возникнет ошибка `TypeError`

```
list_1 = [1, 2, 3]
```

```
print(list_1[1])
```

Вывод: 2

Срезы (1 из 3)

- Срезы также доступны только для индексированных коллекций
- Существует 3 формы:
 - `str[i]` – получение одного элемента
 - `str[a:b]` – получение среза, состоящего из элементов с индексами от **a** включая, до **b** не включая
 - `str[a:b:step]` – получение среза, состоящего из каждых **step** элементов из диапазона от **a** до **b**

```
list_1 = ['Н', 'е', 'л', 'л', 'о']
```

```
print(list_1[2]) Вывод: "л"
```

```
print(list_1[0:3]) Вывод: ['Н', 'е', 'л']
```

```
print(list_1[0:5:2]) Вывод: ['Н', 'л', 'о']
```

Срезы (2 из 3)

📶 Можно создавать именованные срезы

- ⦿ Используется функция `slice()`

```
date = (30, 12, 2020, 16, 47, 30)
```

```
short_date = slice(0, 2)
```

```
full_date = slice(None, 3)
```

```
time = slice(3, None)
```

```
print(date[short_date]) Вывод: (30, 12)
```

```
print(date[full_date]) Вывод: (30, 12, 2020)
```

```
print(date[time]) Вывод: (16, 47, 30)
```

`slice(None, 3)` –
аналог `[:3]`

`date[short_date] =`
`date[0:2]`

Срезы (3 из 3)

📡 При помощи срезов можно менять исходную коллекцию

```
list_1 = [1, 2, 3, 4, 5]
list_1[1:3] = [100]
print(list_1)
Вывод: [1, 100, 4, 5]
```

2 элемента заменены на 1

📡 Число замещаемых элементов может быть любым

```
list_1 = [1, 2, 3, 4, 5]
list_1[1:3] = [100, 200, 300]
print(list_1)
Вывод: [1, 100, 200, 300, 4, 5]
```

2 элемента заменены на 3

Преобразования типов (1 из 2)

- Для преобразования достаточно передать одну коллекцию в качестве аргумента в функцию создания другой коллекции

```
list_1 = [1, 2, 3]
tuple_1 = tuple(list_1)
print(tuple_1)      Вывод: (1, 2, 3)
```

- При конвертации в множества будьте внимательны:

- Произойдет потеря индексов
- Возможна потеря данных

```
list_1 = [1, 2, 3, 3, 2, 5]
set_1 = set(list_1)
print(set_1)      Вывод: {1, 2, 3, 5}
```

Повторяющиеся значения
утрачены

Преобразование типов (2 из 2)

- Для преобразования в словарь необходимо использовать функцию `zip()`

```
index = (1, 2, 3)
letters = ['a', 'b', 'c']
for i in zip(index, letters):
    print(i)
```

Вывод:

```
(1, 'a')
(2, 'b')
(3, 'c')
```

```
dict_1 = dict(zip(index, letters))
print(dict_1)
```

Вывод: {1: 'a', 2: 'b', 3: 'c'}

Модуль collections

- 📡 Модуль collections предоставляет дополнительные возможности при работе с коллекциями
- 📡 Основные инструменты:
 - ⦿ namedtuple – кортеж, в котором доступ к элементам осуществляется через имя, а не индекс
 - ⦿ deque – списки с бóльшим количеством возможностей
 - ⦿ ChainMap – позволяет использовать несколько словарей, как один
 - ⦿ Counter – позволяет быстро и удобно проводить различные подсчеты в коллекциях
 - ⦿ OrderedDict – упорядоченный словарь
 - ⦿ defaultdict – позволяет обращаться к элементам словаря до их добавления